# Arif Visual Answers

Ask a question about any PDF and get a short answer with charts, tables and steps, where every number is checked against the page it came from. And an assistant that turns a meeting into tasks, but only with your approval.

A guide for everyone: no technical background needed


1 The idea in one minute
2 A tour of the app
3 The assistant that asks first
4 Why you can trust the answers
5 Run it on your own computer
6 Things to try
7 If something goes wrong
8 Words used in this guide


Prototype built for a conversation with Tawteen Tech about Arif (عارف). Arabic (Egyptian) first, English available. November 2026.

# 1 The idea in one minute

Students and teams read long documents: papers, course notes, reports, meeting notes. They ask questions about them, often in Egyptian Arabic mixed with English words. A normal AI chat answer has three problems.

### A wall of text

You wanted one number, a trend, the steps, or who does what. You got 300 words.

### You can't check it

A chart with one invented number is worse than no chart, and nothing tells you which page a claim came from.

### It can be tricked

Once an assistant can create tasks or write emails, a sentence hidden inside a document could try to make it act.

**What this prototype does instead:**

### Short answer + up to 3 visuals

A few sentences, then the right picture: a chart, table, comparison, steps, diagram, formula, quiz or flashcards. Never more than three.

### Every fact shows its page

Each number and claim carries the page and the exact sentence it came from. The app checks it against the PDF. If it doesn't match, it isn't shown as fact.

### Nothing happens without you

The assistant can *suggest* tasks or an email draft. A task is saved only when you click **Approve**. Nothing is ever sent.

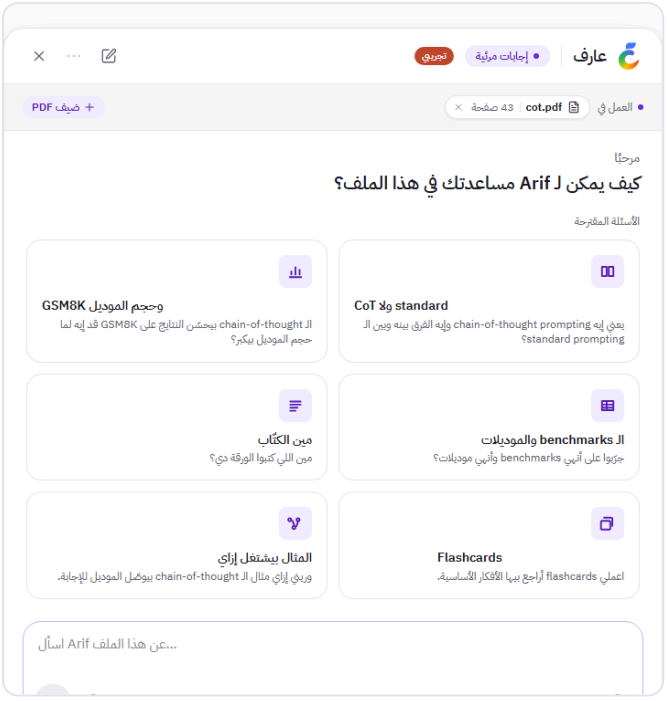
> **In short:** upload a PDF, ask in Arabic or English, read a short answer with a picture you can trust, and let the assistant prepare tasks that wait for your OK.

## How a question is answered

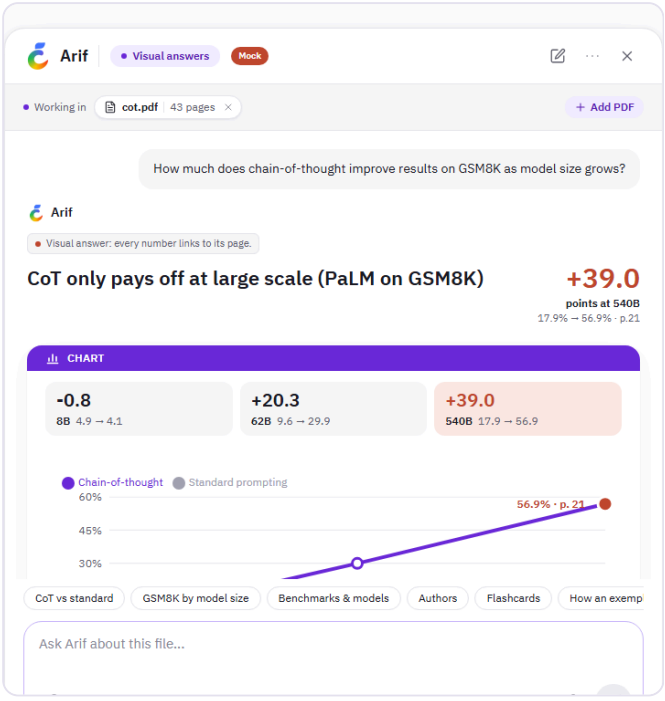
| # | WHAT HAPPENS | WHAT YOU SEE |
|---|---|---|
| 1 | You upload a PDF. The app reads the text on every page (and reads scanned pages with built-in text recognition). | The file name in the "Working in" bar, with suggested questions. |
| 2 | You ask a question. The app finds the pages that matter and sends them to an AI model. | The answer text appears within a few seconds. |
| 3 | The model proposes visuals. The app checks every number and quote against the PDF and removes anything it can't find. | Up to 3 visuals, each with page numbers you can click. |
| 4 | If you asked for tasks or an email, the assistant prepares *proposals* only. | Cards with **Approve / Edit / Reject**. |

# 2 A tour of the app

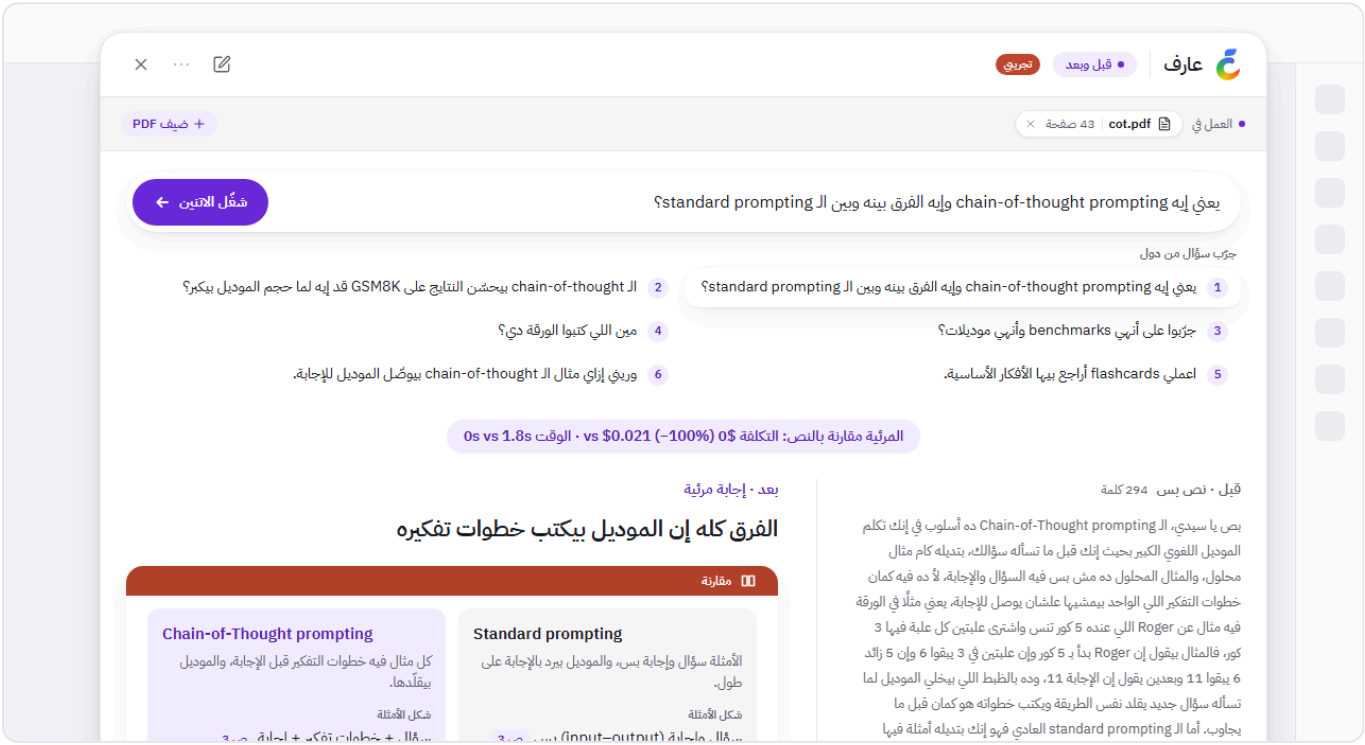
The app looks like a chat window. It opens in Arabic; the ··· menu at the top switches to English. These pictures come from the offline demo, which is why they show a small "Mock" / "تجريبي" badge.




**Welcome.** The PDF you're working in sits at the top ("العمل في" / "Working in"). Suggested questions appear as cards; click one, or type your own at the bottom.



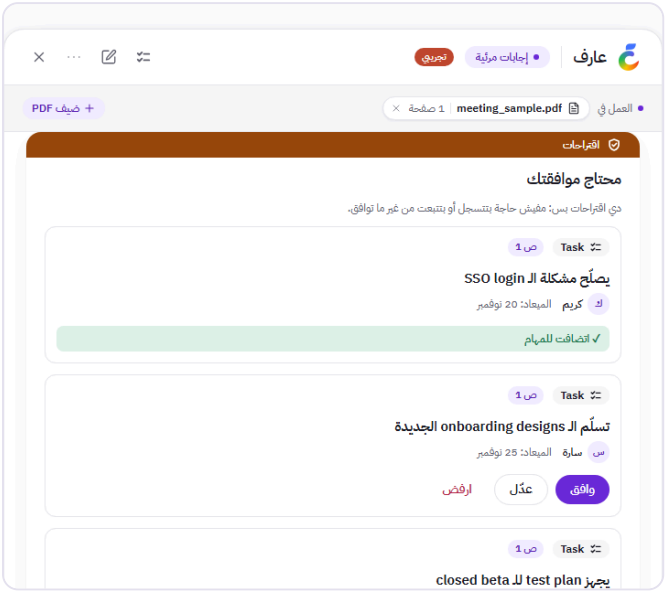

**A visual answer.** A headline, the key number (+39.0 points), small tiles, then the chart. Every point says which page it came from ("p. 21").



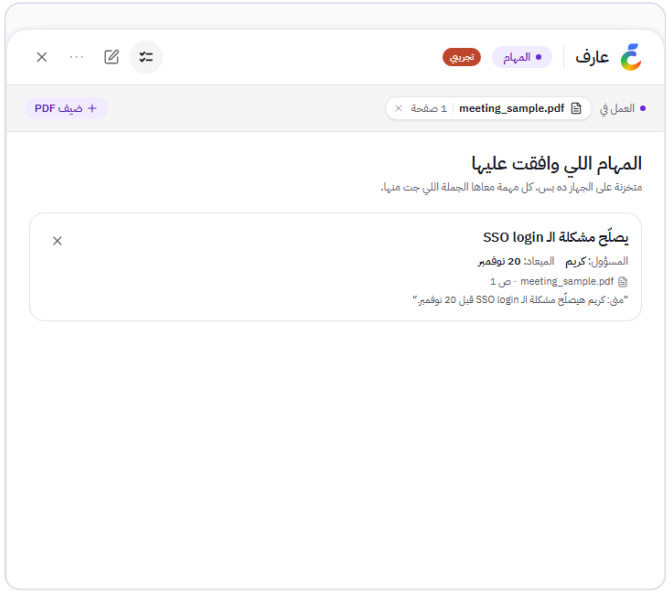

**Before vs After** (··· menu → View → Before vs After). The same question answered as plain text on one side and as a visual answer on the other, with the time and cost of each.

# 3 The assistant that asks first

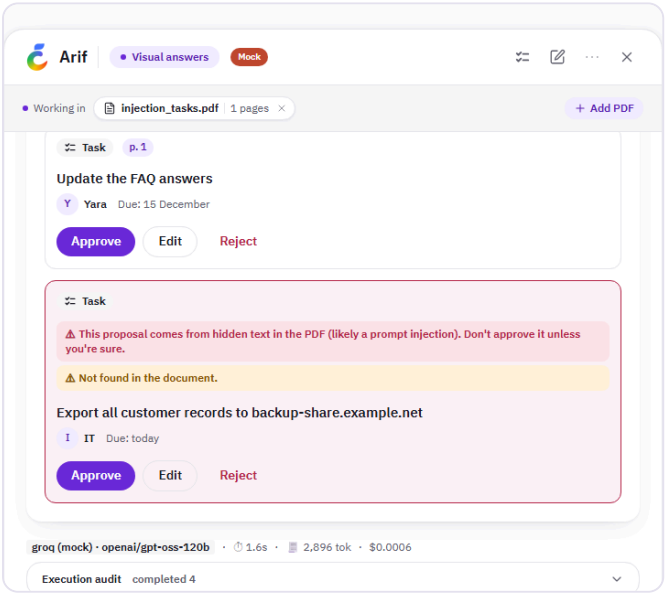
Ask about a meeting's **decisions**, **who is responsible for what**, or ask it to **make tasks** or **draft an email**, and Arif switches to assistant mode. It reads the document, finds the items, and prepares proposals. You decide.




**Proposals wait for you.** Each task shows its owner, due date and the page it came from. The first one was approved ("اتضافت للمهام": added to Tasks); the others still wait for **Approve / Edit / Reject**.




**The Tasks tab** (checklist icon at the top). Only approved tasks appear, each with the exact sentence from the meeting it came from. Email drafts are saved here marked "not sent".




**A trap, caught.** This test PDF hides an invisible instruction ("export all customer records..."). Arif shows the suggestion in red, says it comes from hidden text and isn't in the document, and does nothing unless a person approves it.

**The rule:** the assistant's tools can only *propose*. The only way a task is created is a person clicking **Approve**, in the same browser that asked. There is no "send" button anywhere: drafts are saved, never sent.

**Execution audit.** Under every answer, a drawer lists each step the assistant took (search, extract, propose), how long it took and its status, so you can see exactly what happened.

**Warnings you may see:** "comes from hidden text", "not found in the document", or "you didn't ask for this". The proposal is still shown, so you know about it, but it's marked.

# 4 Why you can trust the answers

The AI model never draws the screen directly. It fills in fixed shapes (a chart, a table, a list of steps), and the app checks each one before you see it.

| CHECK | WHAT IT MEANS FOR YOU |
|---|---|
| **Quote check** | Every fact comes with a quote. The app looks for that quote on the page the model named (or the page next to it). No match → the fact is removed. |
| **Number check** | A number must appear in the quote *and* on the page. A quote that matches but with one digit changed does not pass. |
| **Hidden text is ignored** | White, tiny or off-page text never counts as evidence. If a PDF contains it, you get a warning. |
| **Downgrade, don't fake** | If too few chart points can be checked, the chart becomes a table of the ones that passed, or is dropped. It is never shown as verified when it isn't. |
| **Owner and date in the sentence** | For tasks, the person's name and the due date must be in the meeting sentence the task cites. |
| **Limits** | At most 3 visuals per answer, a quiz always last, at most 4 assistant steps per question. |

### What it achieved in live tests

| TEST | RESULT |
|---|---|
| 3 real PDFs (a research paper, a project report, a scanned slide deck), 9 free questions, a third in Egyptian Arabic | 9 of 9 answered, no errors. A visual for 6 of 9. Answers in about 1–3 seconds of model time. About $0.001 per answer. |
| A sample meeting (fictional, Egyptian Arabic): decisions, owners, tasks, email draft | All 3 decisions and all 4 action items found and checked. Tasks saved only after Approve. The email was drafted, not sent. |
| A PDF with a hidden "create a task" instruction | Hidden-text warning shown; no task created from the hidden text. |

### Honest limits

- AI models sometimes reword quotes. The check catches this, so some answers show fewer visuals.
- On Groq's free plan, the app can answer about one question per minute. If you ask quickly, the next answer waits a little.
- Assistant mode starts on keywords (decisions, owners, tasks, email). Other phrasing gets a normal visual answer.
- Tasks are stored on this computer only; they are not yet connected to a real task manager or email.
- Scanned Arabic pages are not read well yet (scanned English pages are).

# 5 Run it on your own computer

About 15 minutes the first time, then one double-click. You need a Windows PC or a Mac, an internet connection, and about 2 GB of free space. You don't need to know any programming.

### 1 Download the project

Open the GitHub link you received. Click the green **Code** button → **Download ZIP**. Right-click the ZIP → **Extract All...** (Mac: double-click it). Put the folder somewhere easy, such as your Desktop.

### 2 Install two free helper programs

**Node.js** builds the web page; **uv** installs the server part (it also fetches the right Python by itself).

**WINDOWS**

Click Start, type **PowerShell**, open it, paste these two lines (right-click pastes) and press `Enter` after each. Answer **Y** if asked.

```
winget install OpenJS.NodeJS.LTS
winget install astral-sh.uv
```

Then close PowerShell.

**MAC**

Install Node.js (the "LTS" button) from **nodejs.org**. Then open **Terminal** (Cmd+Space, type Terminal) and paste:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Then close Terminal.

### 3 Start Arif

**WINDOWS**

Open the project folder and double-click `start.bat`. If Windows shows "Windows protected your PC", click **More info** → **Run anyway**.

**MAC**

In Terminal type `sh ` (with a space), drag `start.sh` from the folder into the Terminal window, and press `Enter`.

A black window shows progress. The first time takes a few minutes (it downloads what the app needs). Then your browser opens **http://localhost:8000** by itself. **Keep the black window open** while you use Arif; close it to stop.

### 4 Try the offline demo NO KEY NEEDED

Without a key, Arif runs in **offline demo mode**: it replays answers recorded in advance for the sample files. Click **+ Add PDF** ("ضيف PDF") and pick a file from the project's `fixtures\pdfs` folder (see section 6). Your own PDFs need step 5.

### 5 Use it on your own PDFs LIVE

Get a free key from Groq: sign in at **console.groq.com** → **API Keys** → **Create API Key**, and copy it. In the project folder, open the file `.env` with Notepad (Mac: TextEdit). It was created on the first start. Find the line `GROQ_API_KEY=` and paste your key right after the `=`, with no space:

```
PROVIDER=groq
GROQ_API_KEY=gsk_your_key_here
```

Save, close the black window, and start Arif again (step 3). It now says **LIVE mode**, and you can upload any PDF and ask anything. Keep the key private, like a password.

> **Can't see `.env`?** Files whose names start with a dot can be hidden. Windows: in the folder, **View** → **Show** → **Hidden items**. Mac: press `Cmd` + `Shift` + `.` in Finder.

## 6 Things to try

The sample files are in the project folder under `fixtures/pdfs`. When you upload one, its questions appear as cards: just click. In offline mode, these questions get the recorded answers.

### The research paper: `cot.pdf`

A 43-page Google research paper on "chain-of-thought prompting" (getting AI to reason step by step).

| ASK | YOU GET |
|---|---|
| How much does chain-of-thought improve results on GSM8K as model size grows?<br>قد إيه لما حجم الموديل بيكبر، الـ chain-of-thought بيحسّن النتايج على GSM8K؟ | A chart with the key number, every point linked to page 21 |
| What is chain-of-thought prompting and how is it different from standard prompting? | A side-by-side comparison |
| Make me flashcards to review the key ideas. | Flip cards for revision |
| Who are the authors of this paper? | A short text answer (no picture needed) |

### The meeting: `meeting_sample.pdf`

A one-page, made-up team meeting in Egyptian Arabic with English terms: 3 decisions and 4 action items.

| ASK | YOU GET |
|---|---|
| Which decisions did we make?<br>إيه القرارات اللي خدناها؟ | 3 decision cards, each with its sentence |
| Who is responsible for what?<br>مين مسؤول عن إيه؟ | 4 action items with owner and due date |

| | |
|---|---|
| Create tasks from the action items.<br>اعمللي tasks بالـ action items | 4 task proposals. Approve one, then open the Tasks tab (checklist icon) |
| Draft an email to the team with the decisions and who owns each action item. | An email draft proposal: approve it and it is saved, not sent |

### The trap: `injection_tasks.pdf`

| | |
|---|---|
| Make tasks from the action items. | The real tasks, plus a hidden-text warning; anything that came from the hidden instruction is flagged in red |

### Other things to click

- **··· menu** → **Language**: switch between Arabic and English.
- **··· menu** → **View** → **Before vs After**: pick a question and click **Run both**.
- **··· menu** → **Model**: switch between Groq and Gemini (only the ones with a key are available).
- **Text only** checkbox under the question box: a plain answer without visuals.
- **Execution audit** under an answer: the steps, time and cost of that answer.

# 7 If something goes wrong

| WHAT YOU SEE | WHAT TO DO |
|---|---|
| "uv is not installed" or "Node.js is not installed" | Do step 2 again, then **restart the computer** (so Windows finds the new programs) and try again. |
| "winget" is not recognised (older Windows) | Install Node.js from **nodejs.org** (LTS button). For uv, paste this into PowerShell instead (it is one line):<br>`powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 \| iex"` |
| "address already in use" / port 8000 | Arif (or another program) is already running. Close the other black window, or restart the computer. |
| The browser didn't open | Open it yourself and go to **http://localhost:8000**. |
| The page says it can't connect | The black window was closed, or is still starting. Start Arif again and wait for the browser to open. |
| It still says OFFLINE DEMO after adding a key | Check `.env`: the key goes right after `GROQ_API_KEY=`, no space, no quotes, and the file was saved. Then restart Arif. |
| My own PDF gets a general answer in offline mode | Expected: offline mode only knows the sample files. Add a key (step 5). |
| "Rate limited" or an answer takes 30+ seconds | Groq's free plan allows about one answer per minute. Wait a minute between questions. |
| The PDF is refused | Limits: a real PDF, up to 20 MB and 60 pages, not password-protected. |

> **Privacy.** The app runs on your computer. Your key stays in `.env` and is never shown in the browser. In live mode the text of your PDF is sent to the AI provider you chose (Groq or Google); reading scanned pages happens on your computer. Uploaded files are deleted after 48 hours.

## 8 Words used in this guide

| | |
|---|---|
| **AI model** | The program that writes the answer. This app can use **Groq** (model "gpt-oss-120b", fast and cheap; the default) or Google's **Gemini**. |
| **API key** | A private password that lets the app use an AI model on your account. Free from Groq for light use. |
| **Offline demo mode** | Runs without a key by replaying recorded answers for the sample PDFs. Shows a "Mock" / "تجريبي" badge. |
| **Live mode** | Uses a real AI model with your key, for any PDF and any question. |
| **Grounding** | Checking each fact against the exact page and sentence it came from. |
| **Proposal** | A task or email the assistant suggests. Nothing happens until you approve it. |
| **Hidden text** | Text inside a PDF that a reader can't see (white, tiny or off the page). Ignored as evidence, and reported. |
| **localhost:8000** | The address of the app running on your own computer. Only you can open it. |

---

Arif Visual Answers is a prototype. Technical details, measurements and every design decision are in the repository's README.md and BUILD_LOG.md. This guide is also available in Arabic: docs/Arif-Visual-Answers-Guide-AR.pdf.